ICT374 Lab 7:
Shell Command Separators; Parse a Command Line into a Sequence of Commands; Unnamed Pipe, Blocking of Unnamed Pipes and Deadlocks due to Unnamed Pipes, and FIFO; Control the access to shared resources using semaphores; Exchange messages using message queues and exchange data using shared memories.

Learning Objectives:

1. Understand different shell command separators icluding pipe "|", background execution "&", and sequential execution ";".
2. Be able to parse a command line into a sequence of commands.
3. Be able to use unnamed pipe to communicate with processes with a common ancestry.
4. Understand when reading from a pipe could lead to process suspension (blocking).
5. Understand when writing to a pipe could lead to process suspension (blocking).
6. Understand what could cause deadlock in a system involving several process and pipes and be able to identify deadlocks.
7. Understand the System V IPCs and be able to create and remove these IPCs from the command line.
8. Understand the general programming interface of the System IPCs and be able to create these IPCs in C programs.
9. Understand and be able to use message queues to exchange messages among different processes.
10. Understand why access control is required multiple processes access the same shared resource and be able to use a binary semaphore to enforce mutual exclusion.
11. Understand the Producer-Consumer Problem and be able to use several binary semaphores to sychonise the Producer process with the Consumer process.
12. Understand the shared memory mechanism and be able to create shared memory and use it to exchange data between multiple processes. Understand when mutual exclusion is required and when synchronisation is required and know to implement these with one or several semaphores.

Required Reading:

- Lecture Notes for Topic 7
- Stevens and Rago: Ch 15
- Functions in `string.h`.

C Programming Exercises

Note, all exercises must be done via the terminal commands, not GUI tools. Do not use IDE to build your programs. Do not use GUI file managers to manage your directories and files. Also each exercise should be in a separate directory, which itself, is under a directory called `lab08`

1. **Shell command separators**

   A Unix shell can process a command line that consists of more than one command. That is, the commannd line

   ```
   %  ls -l /dev | grep tty > ttylist &  ps -f > plist ; ls -l *list
   ```

   consists of the four commands:

   ```
   1)  ls -l /dev
   ```

```
2)  grep tty > ttylist
3)  ps -f > plist
4)  ls -l *.list
```

These four commands are connected, or separated, by three special symbols:

- **Pipe "|"**

  the first two commands are connected by a pipe. They run as a background job. The special symbol "|" means that the shell will create an unnamed pipe to link the standard output of the first process with the standard input of the second process.

- **Concurrent execution "&"**

  The third command is ended with a "&". This means the shell does not wait for the proceeding command

  ```
  grep tty > ttylist
  ```

  to finish before starting the next command

  ```
  ps -f > plist
  ```

  The symbol "&" can also be at the end of a command line, which means the shell does not wait for the proceeding command to finish before printing a shell prompt to invite the user to type the next command line.

- **Sequential execution ";"**

  If a command is followed by symbol ";", it means the shell must wait for the command to finish before executing the next command. Just as with "&", the symbol can be the last symbol in a command line. It means that the shell has to wait for the command before ";" to finish before printing the next shell prompt to allow the user to type in the next command line.

Therefore, these special symbols "|", "&", and ";" are command separators.

Some of the commands listed contains standard output redirection ">". The special symbols, "<" (for standard input redirection), ">" (for standard output redirection) and "2>" (for standard error redirection), are not command separators. For instance, in command

```
grep tty > ttylist
```

the clause "> ttylist" is a part of the command. However this part is not fed into the command during `exec` call, rather the shell will set up standard output redirection to file "ttylist" before loading and running the command "grep tty" with the `exec` call.

2. **Separating a command line into a sequence of commands**

   This exercise is relevant to Project 1 - A Simple Unix Shell.

   Continue from C Programming Exercise 3 in Lab 6, modify the program from that exercise so that your program now breaks up a command line into a sequence of commands. Instead of printing out a sequence of tokens from the command line, your program should now print out a sequence of commands and for each command, it prints out a sequence of tokens contained in that command.

   You may want to consider a new module named `command`, where the module header defines the following:

   ```
   struct CommandStructure {
   ```

```
    int first;          // index of the first token of the command in the array of tokens
    int last;           // index of the last token of the command in the array of tokens
    char* separator;    // the command separator that follows the command. It should be
                        // one of the following
                        //  "|"   - pipe  to the next command
                        //  "&"   - shell does not wait for this command
                        //  ";"   - shell wait for this command
};

typedef struct CommandStructure  Command;  // type name for the command structure

// take a list of tokens and break it into an array of commands
int separateCommands(char *tokens[], Command command[]);
```

You may use the following code as the starting point for this exercise: `command.h` and `command.c`. Please note this module is not tested at all, so it is guaranteed to have bugs in it.

Make sure that you modify the makefile to take into account the module `command`.

Unix Programming Exercises

1. **Unnamed pipe**

   Write a program that creates a child process. The child process sends a message to the parent and then terminates. Once the parent receives the message from the child, it prints the message and also terminates (use an unnamed pipe).

2. **Blocking read from a unnamed pipe**

   Modify the program in Exercise 1 above, so that the child process sleeps for 10 seconds after it has sent out the message, then closes the write end of the pipe and reports that it has closed the pipe. It then sleeps for another 10 seconds before it terminates. The parent process should terminate as soon as the child process has closed its ends of the pipe (but before it terminates).

3. **Finding deadlocks due to pipe connections**

   Referring to the example on pages 8 to 11 of the Topic 7 Lecture Notes (Interprocess Communications), explore the behaviours of that program by removing the following `close` calls:

   - close(BA[1]) at line L1;
   - close(AB[1]) at line L2;
   - close(AB[1]) at line L3.

   The program is listed below:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <sys/types.h>
#include <unistd.h>

char *mesg="Hello, world!";

/* convert upper case to lower case or vise versa */
char conv(char c)
{
     if (islower(c)) return toupper(c);
     if (isupper(c)) return tolower(c);
     return c;
}

main()
{
```

```
        pid_t pid, AB[2], BA[2];
        char buf[20];

        /* create two pipes */
        if (pipe(AB)<0 || pipe(BA)<0)
            perror("pipe calls"), exit(1);

        /* create process B */
        if ((pid=fork())<0)
            perror("fork call"), exit(2);


        if (pid>0) {       /* process A */
           close(AB[0]); close(BA[1]);                // L1

           printf("     A sends: %s\n", mesg);
           write(AB[1], mesg, strlen(mesg));
           close(AB[1]);                              // L2

           printf("A receives: ");
           while (read(BA[0], buf, 1)==1)
                printf("%c", buf[0]);

           printf("\n"); exit(0);
        }

        if (pid==0) {    /* process B */
           close(BA[0]); close(AB[1]);                // L3

           while (read(AB[0], buf, 1)==1) {
               buf[0] = conv(buf[0]);
               write(BA[1], buf, 1);
           }
           exit(0);
        }
    }
```

For each experiment, you should use the `ps` command to check which process has exited and which one has not. Explain the reason why the program behaves in that way.

4. **Creating a FIFO from command line**

   Create a FIFO from the command line. Check its size and access permissions using command `ls`. Also check your current umask value (command `umask`) to explain why the FIFO has got that specific set of access permissions.

5. **Using FIFO for communications between programs**

   Modify the program in Exercise 2, so that it uses the FIFO created in Exercise 4 above for communications (rather than using an unamed pipe).

6. **Using a FIFO to communicate between two terminals.**

   Create a FIFO `~/my.fifo`. Start two shell terminals. On one terminal, type the command

   ```
   cat > ~/my.fifo
   ```

   and on another terminal, type

   ```
   cat < ~/my.fifo
   ```

   When you type something in the first terminal, can you see them in the second terminal? Explain the reason.

7. **Finding existing System V IPCs**

On your Linux machine, check whether there is any System V IPC using the command `ipcs`. In my test using Redhat 9, Mozilla created a share memory IPC. Once you have found an IPC, you may use `ipcs -p` to find the creator process of that IPC.

8. **A simple chat program using message queue**

   Write a simple chat program using Message Queues that allows two persons to exchange messages with each other on the same computer.

   Hints: refer to the example on pages 20-25 in Lecture Notes for Topic 7.

9. **Collecting marks from another process via a shared memory.**

   Write a program to ask user to input student marks, one at a time, until the user types "end". Each time the program gets a mark, it passes it to another process via a shared variable before it goes on to ask for the next mark from the user.

   Also write another program to collect these marks from the first program via the same shared variable. Once the user input finishes, the second program displays the average mark on the screen.

   Hints: refer to the example on pages 43-45 in Lecture Notes for Topic 7.

The Weekly Lab

Submit the following exercises:

1. C Programming Exercise 2.
2. Unix Programming Exercise 2.
3. Unix Programming Exercise 5.
4. Unix Programming Exercise 9.

This weekly lab must be submitted to the LMS. The deadline of submission is given in the Unit Information Page of the LMS. You must adhere to the instructions in the document titled The Requirements for Documenting and Submitting Weekly Labs when preparing and submitting a weekly lab.

Dr Hong Xie
Unit Coordinator

Last update: Thursday, 18 September 2025

Document author: Dr. H. Xie, Unit Coordinator, ICT374 Operating Systems and Systems Programming
Last Modified: Thursday, 18-Sep-2025 16:09:54 AWST
Disclaimer & Copyright Notice © 2026 Murdoch University
This document is relevant for 2025 only